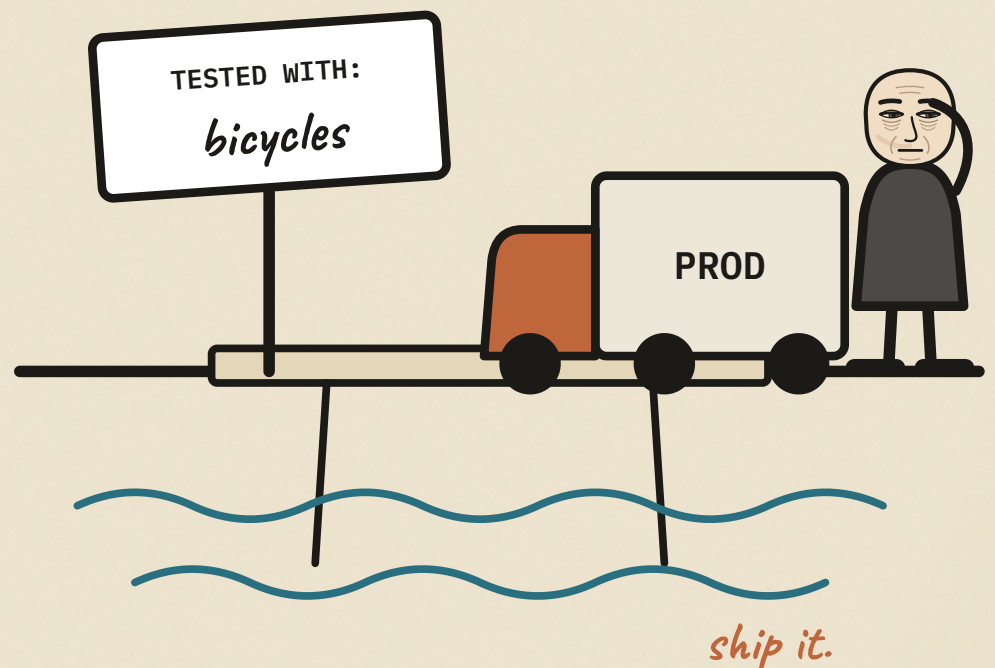


A STORYBOARD IN 13 FRAMES

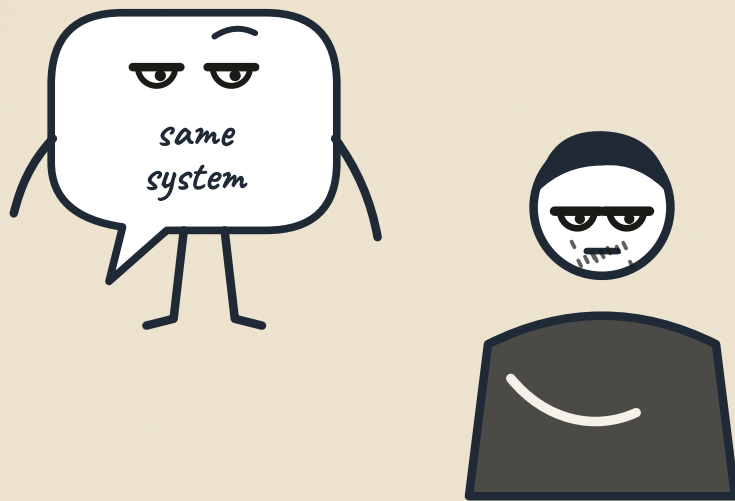
When is an AI system ready for real use?

*Deployed. Delighted.
Disowned.*

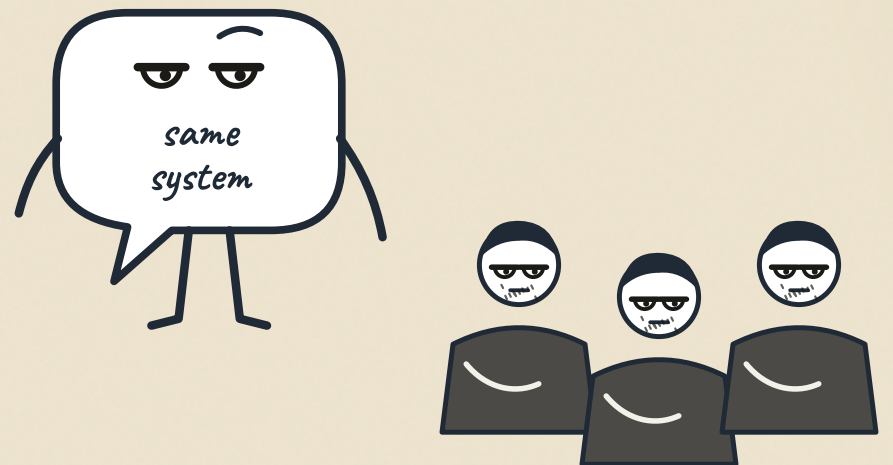


Deployment means it's running. Readiness means the evidence and controls justify this exposure: these users, doing this job, with these consequences when it's wrong.

Ready for what, exactly?



a team reviewing it



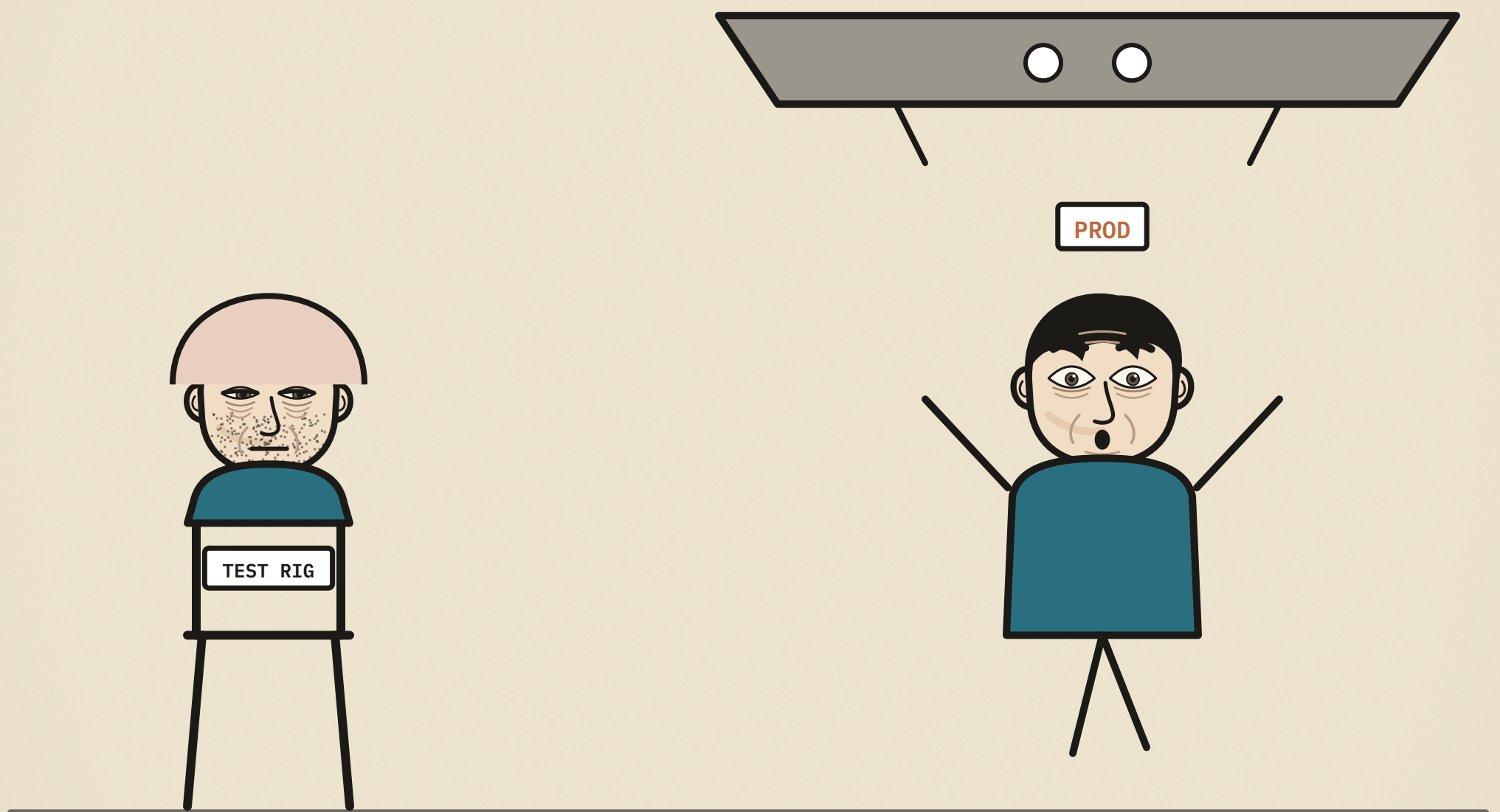
customers acting on it



same system. different exposure.

Name the exposure precisely: which users, doing what, with what riding on the result. That turns an unanswerable question into one with an answer.

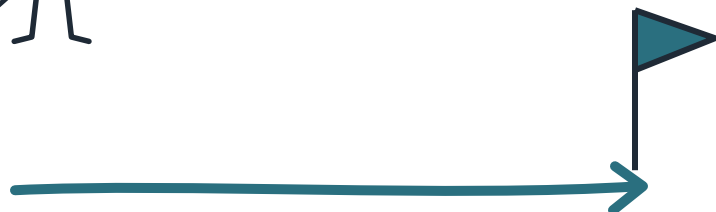
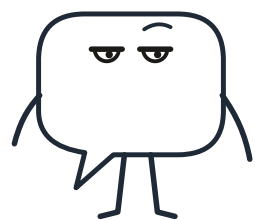
Ready for which drop?



Tested from a chair. Deployed from a plane.

A system is not ready in general. It is ready, or not, for a particular exposure to a particular use.

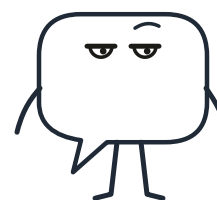
The demo proved it's possible.



the happy path

The demo

*one user, one path,
little at stake*



malformed input

slow upstream

an odd goal

costly failure

Real use

*the cases nobody
demoed*

Readiness asks about conditions the demonstration may never have exercised. Evidence that's silent about a case isn't a pass.

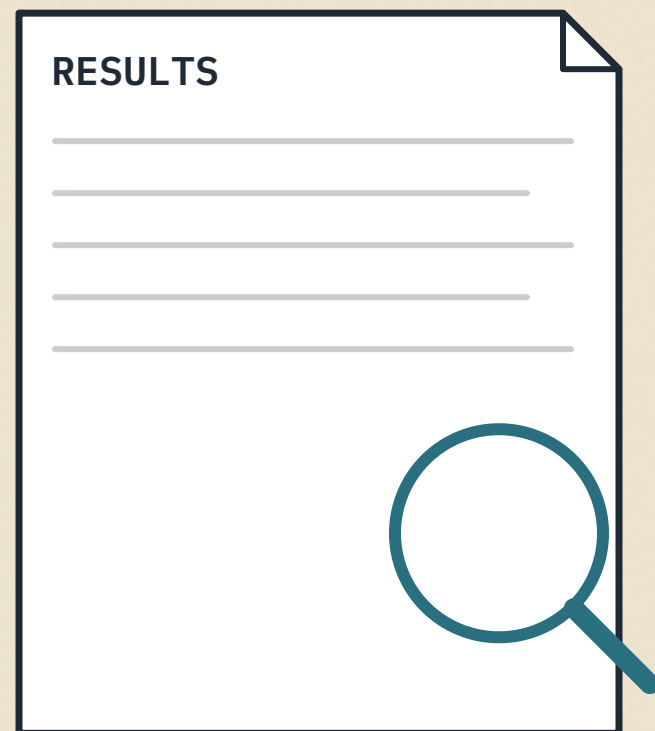
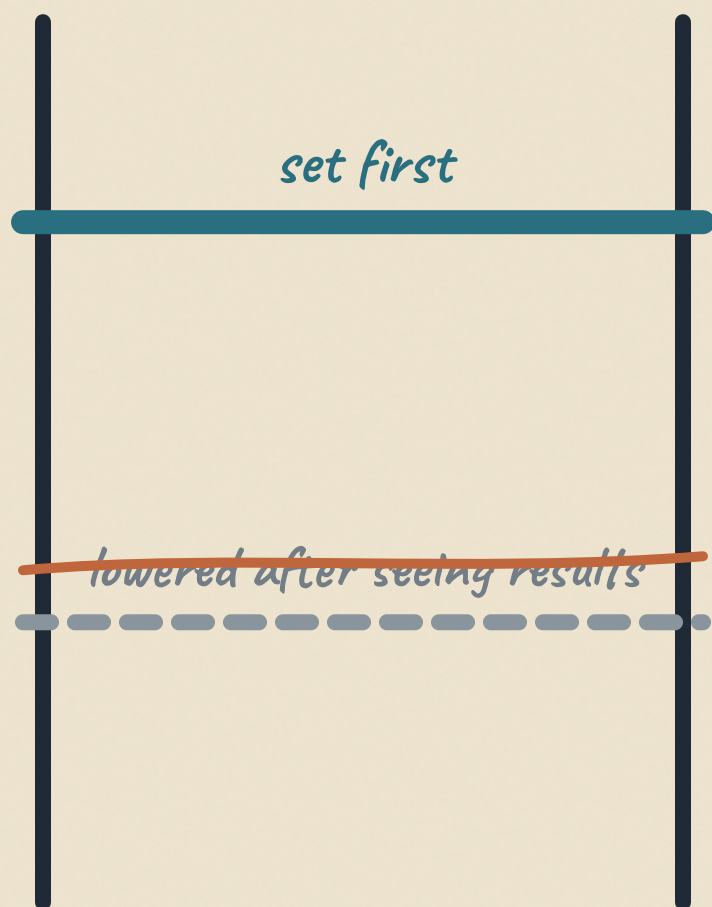
The demo went perfectly.



Twelve rehearsals. One user. One path.

A demonstration compresses the system: one user, one goal, a selected input, one path through, little at stake. It shows the thing is possible, which is a narrower question than whether it is ready.

Set the bar before you look.



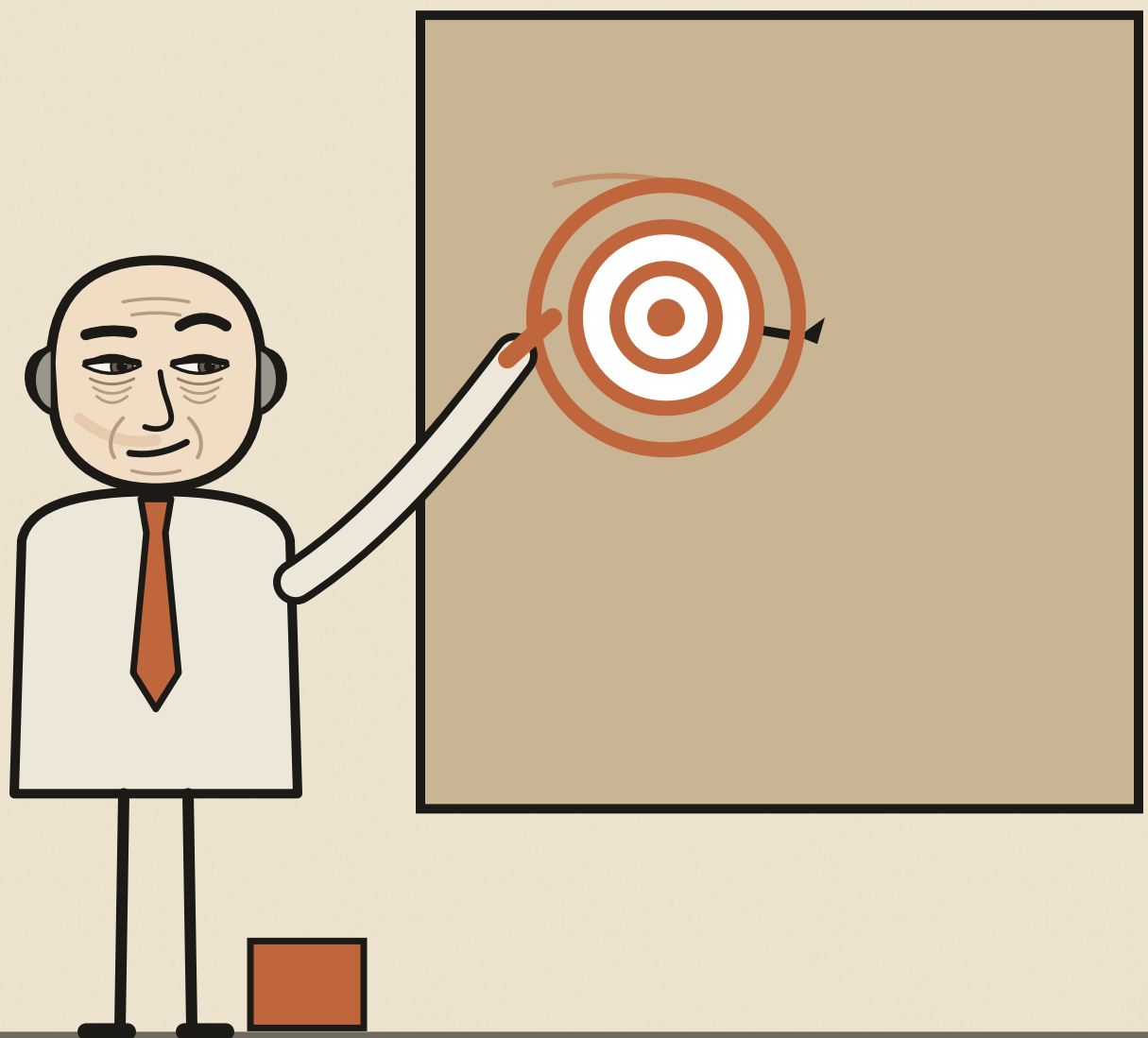
fitting the rule to the results

Define the release criterion against an evaluation set that fits the use and its failure costs, before exposure. Chosen afterwards, it can be fitted to the results.

Hit the target. Then drew it.

(the bow)

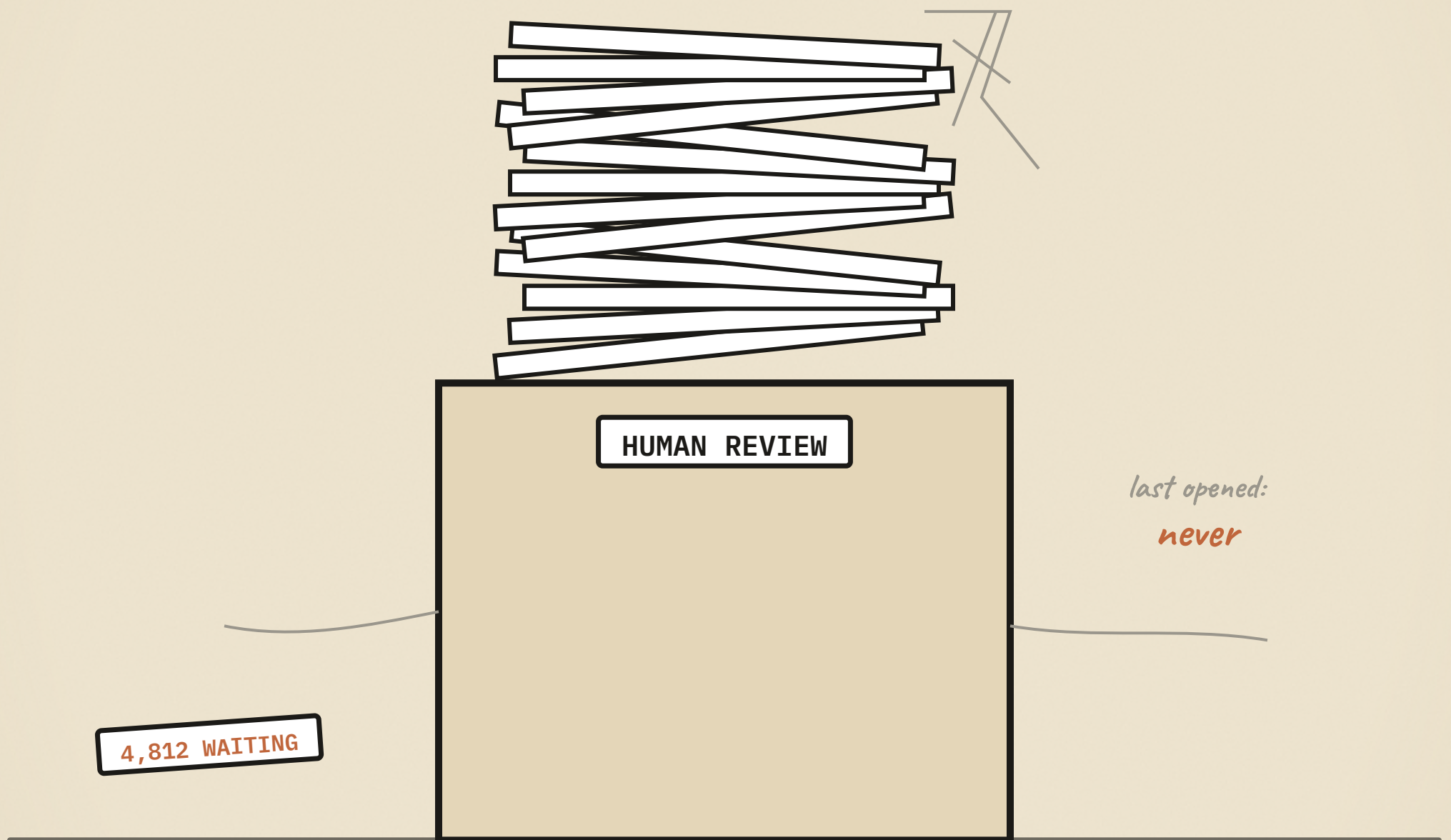
.....



Bullseye. Target painted afterwards.

Define an explicit release criterion against an evaluation set that fits the intended use and its failure consequences, before exposure rather than after.

If it can't continue, then what?



Fallback: a queue. Reader: nobody.

Name the safe alternative that actually exists when the system should not continue: stop and say so, decline, keep the prior state, queue it, restrict it, or hand it to a person who can act. A queue nobody can act on doesn't count.

Some things don't un-send.



*some effects
don't come back.*



Stop it

*before more
happens*

Reverse it

*not a sent message
or a payment*

Correct it

after the fact

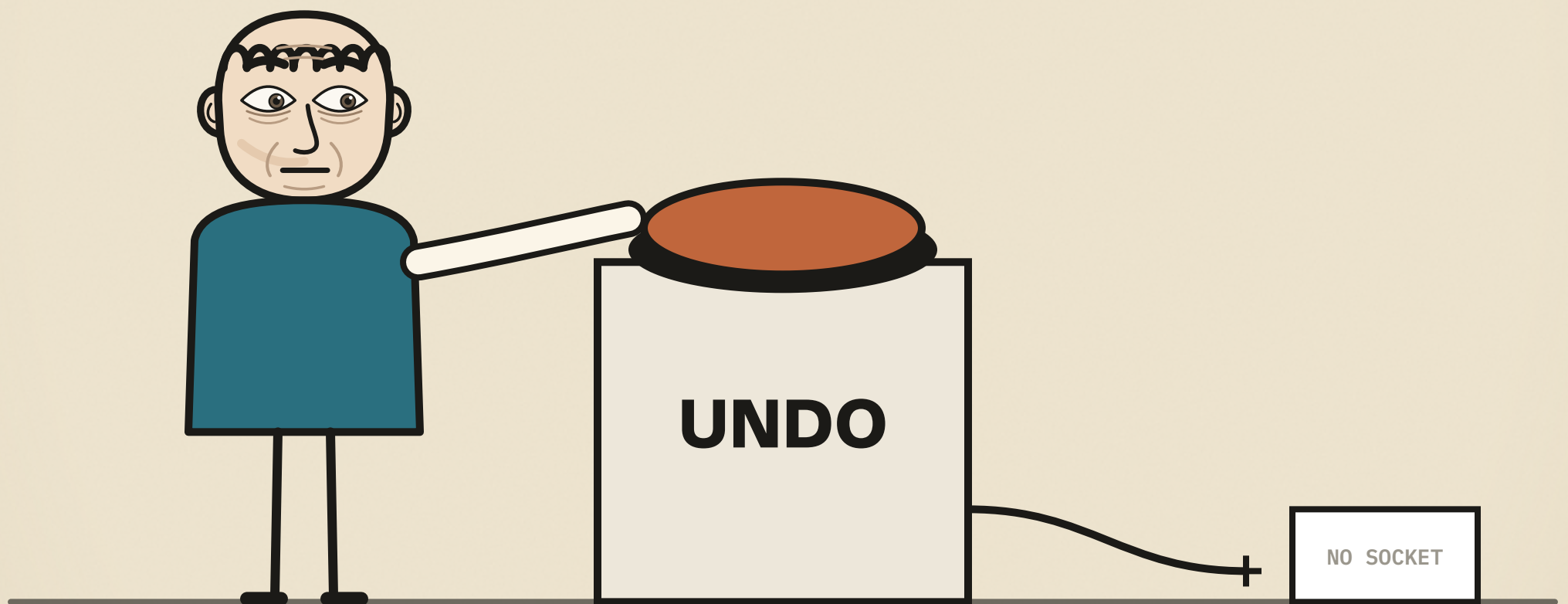
Contain it

*limit how far
it travels*

The question isn't whether a rollback procedure exists. It's what can be stopped, reversed, corrected or contained.

What can actually be undone?

click. click. click.

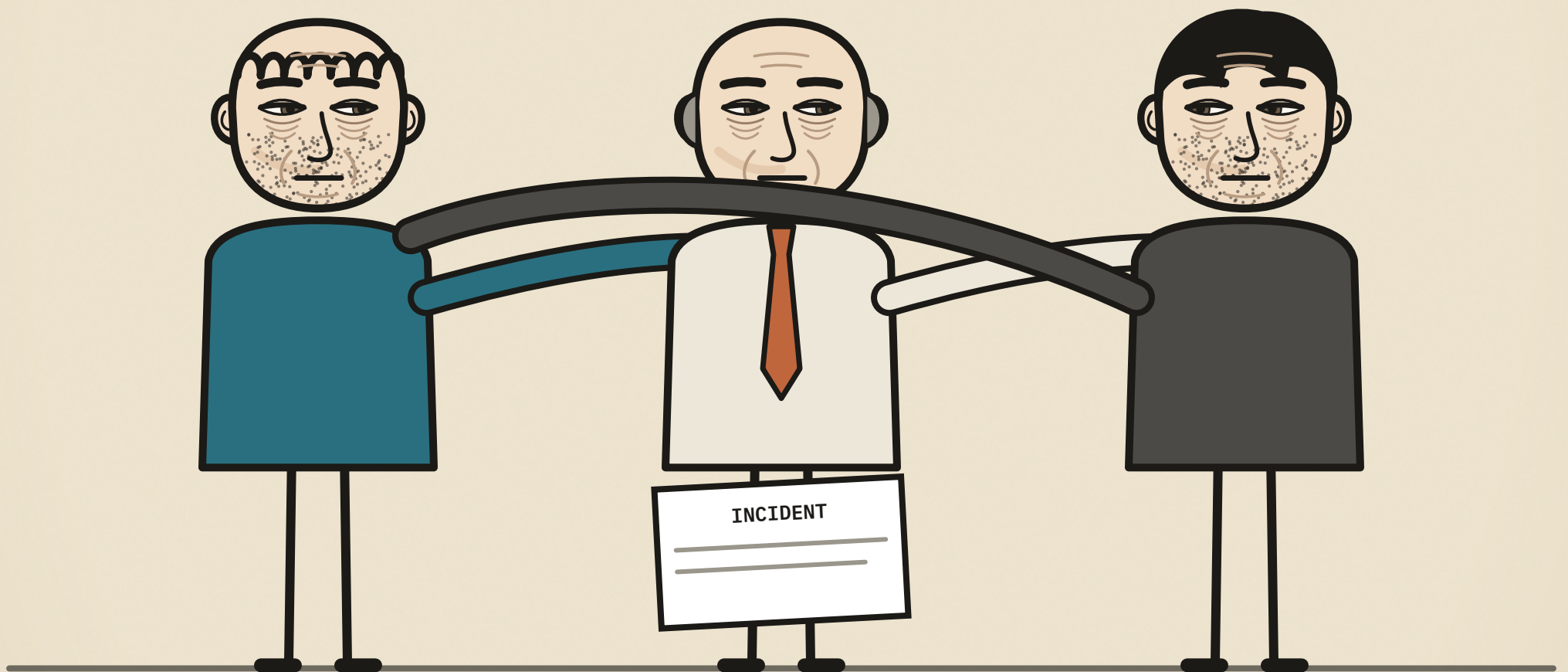


The undo button. Connected to nothing.

Before exposure, name what can really be reversed and how. In this repository, when checks that run after a draft is placed fail, the previous state of the affected files is restored.

Who owns it when it breaks?

OWNER: _____



Everyone points. Nobody's name is on it.

Before exposure, somebody owns the decision to expose, and somebody owns the response when it misbehaves. If nobody can be named for either, that is itself a finding about readiness.

The decision has a shelf life.

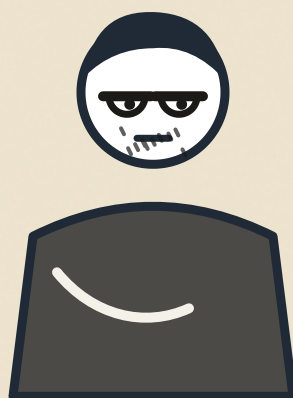


Approved. Best before the next model update.

A change to the model, the instructions, the retrieved corpus, a tool, the workflow or the intended users can undermine part of the basis the release decision rested on.

Three answers, in writing.

- ☐ Evidence against the failures that matter
- ☐ What limits the damage when it's incomplete
- ☐ Who decides, and who answers



named, not assumed

If those can't be answered, nobody can say yet whether it's ready.
That's a smaller problem, and a more fixable one.

Read the whole explanation

[pruningmypothos.com/systems/
from-prompt-to-production/](https://pruningmypothos.com/systems/from-prompt-to-production/)

